



Progress Report 1

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project
Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: February 7, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Prepared By:

<u>Name</u>	<u>Student ID</u>	<u>Email Address</u>
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Work Date/Hours Log.....	2
2. Summary Description of Work Done.....	4
2.1 Environment Setup (Sprint 0).....	4
2.2 Learning & Research: Streamlit Framework	4
Proof of Concept: StatCan Dashboard.....	4
2.3 UI Mockups (Sprint 1A)	5
2.4 Database Setup (Sprint 1C)	6
2.5 Authentication Feature (Sprint 1D).....	6
2.6 Basic Navigation (Sprint 1E)	8
2.7 Issues Encountered and Solutions	8
3. Repo Check-in of Implementation Completed	9
4. Revised Timeline and Next Steps.....	11
4.1 What's Next: Sprint 2 - Symptom Assessment Chatbot	12
5. AI Use Section.....	14
5.1 AI Tools Usage Summary	14
5.2 Table of AI Tools and Specific Use	14
5.3 Statement of Originality.....	15
Appendix A: AI Prompt History	16
Prompt Session 1: Understanding Streamlit Session State.....	16
Prompt Session 2: SQLite Database Setup for Python.....	16
Prompt Session 3: Bcrypt Implementation in Python	17
Prompt Session 4: Streamlit Login/Register UI Pattern.....	17
Prompt Session 5: Page Protection Pattern	17
Prompt Session 6: Claude Code - Database Module Generation	18
Appendix B: UI Mockups	19

1. Work Date/Hours Log

Note: Logs are entered as work is completed with granular entries. Study hours for learning new technologies are included as approved by the instructor.

Date	Hours	Description of Work Done
2026-01-30	1.50	Sprint 0: Environment Setup - Install Python 3.10+, VS Code, create GitHub repository (BC-Health-Platform), create virtual environment
	1.00	Sprint 0: Create OpenAI account with \$5 credit, generate API key; Create Streamlit Cloud account
	1.00	Sprint 0: Install packages (streamlit, openai, pandas, scikit-learn, plotly, folium, bcrypt, nltk, sqlalchemy); Run 'streamlit hello' to verify; Create requirements.txt
2026-01-31	3.00	Learning & Research: Udemy course - Streamlit for Data Science & ML (Sections 1-4: Streamlit basics, components, layouts, data display)
2026-02-01	3.00	Learning & Research: Udemy course - Streamlit for Data Science & ML (Sections 5-8: Charts, maps, ML deployment, interactive dashboards)
	1.50	Proof of Concept: Built sample dashboard pulling StatCan population data; Practiced Streamlit charts, filters, and data caching
2026-02-01	2.00	Sprint 1A: Created UI mockups using UX Pilot AI tool - Login page, Register page, Home Dashboard, Facility Finder, Health Library (5 mockups)
2026-02-02	1.50	Sprint 1A: Created remaining UI mockups - Chatbot Interface, My Dashboard (Personal), System Analytics; Saved all mockups to Misc/design/ folder
2026-02-02	0.75	Sprint 1B: Created project folder structure (src/, pages/, components/, .streamlit/, data/); Set VCH theme colors in config.toml (teal #006271); Created placeholder files for all pages
2026-02-03	0.75	Sprint 1C: Created components/database.py with SQLite connection; Designed users table schema (id, name, email, password_hash, age, gender, region, conditions)
	0.50	Sprint 1C: Designed assessments table schema (id, user_id, symptoms, urgency, recommendation, timestamp)
	0.75	Sprint 1C: Implemented user CRUD functions (create_user, get_user_by_email, get_user_by_id, update_user, verify_user_exists)

	0.50	Sprint 1C: Implemented assessment CRUD functions (save_assessment, get_user_assessments, get_assessment_by_id, get_assessment_stats)
	0.33	Sprint 1C: Tested database with dummy data; Installed SQLite Viewer VS Code extension; Verified tables created correctly
	0.10	Sprint 1C: Git commit and push - 'Sprint 1C complete: Database setup with users and assessments tables'
2026-02-04	0.75	Sprint 1D: Created components/auth.py with bcrypt password hashing (hash_password, verify_password functions); Implemented email format validation and password strength checks (min 8 chars)
	1.00	Sprint 1D: Built Login tab UI in app.py (email input, password with visibility toggle, remember me checkbox) matching UX Pilot mockup
	1.50	Sprint 1D: Built Register tab UI in app.py (full name, email, password, confirm password, age, gender, BC health region dropdown, 10 health condition checkboxes)
	0.75	Sprint 1D: Implemented login logic - verify credentials against database, set Streamlit session_state for authenticated user
2026-02-05	0.75	Sprint 1D: Implemented register logic - validate all fields, hash password with bcrypt, save user to SQLite database
	0.25	Sprint 1D: Added logout functionality - clear session_state and redirect to login page
	0.50	Sprint 1D: Created require_authentication() helper function; Applied page protection to all 6 pages
	0.50	Sprint 1D: Tested full authentication flow (9 test cases); Cleaned up orphaned test data from Sprint 1C; Verified bcrypt hashing in SQLite Viewer
	0.75	Sprint 1E: Verified all 6 navigation pages load correctly (Home, Symptom Assessment, My Dashboard, Facility Finder, Health Library, System Analytics); Tested sidebar navigation
	0.25	Sprint 1E: Git commit and push - 'Sprint 1D complete - Authentication feature with bcrypt hashing, cleaned test data'
	1.00	Documentation: Updated work log; Prepared Progress Report 1; Updated task tracker with Sprint 1D and 1E completion status
TOTAL	26.18	Hours logged for Progress Report 1 (Jan 30 - Feb 5, 2026)

2. Summary Description of Work Done

During this initial reporting period (from January 30 to February 5), I have finished the entire **Sprint 0 (Environment Setup)** and **Sprint 1 (Foundation)** for the BC Personal Health Management Platform. This work involved setting up the development environment, learning the Streamlit framework through a well-organized Udemy course, designing UI mockups, implementing the SQLite database, creating a fully-fledged authentication system with bcrypt password hashing, and setting up multi-page navigation. The next paragraphs elaborate on each area of work.

2.1 Environment Setup (Sprint 0)

The programming environment was set up on January 30, 2026. I chose Python 3.10+ as the main programming language, installed Visual Studio Code as the code editor, and made a GitHub repository (BC-Health-Platform) for version control. A Python virtual environment was set up to separate dependencies from other projects and the necessary packages were installed such as Streamlit (web framework), OpenAI (API integration), pandas and scikit-learn (data processing and ML), plotly and folium (visualization), and bcrypt (password security). The Streamlit framework was tested by executing 'streamlit hello' to ensure a functional installation. An OpenAI user was registered, along with a \$5 API credit, and an API key was created for the symptom assessment chatbot. Also, a Streamlit Cloud account was created in anticipation of future deployment. Additionally, a requirements.txt file was made listing all package dependencies to ensure reproducibility.

2.2 Learning & Research: Streamlit Framework

Before starting implementation, I spent time studying the Streamlit framework through an organized Udemy course ("Streamlit for Data Science & ML"). The reason for choosing Streamlit as the full-stack web framework for this particular project was that it basically integrates frontend UI rendering and backend Python logic into one single framework, thus, there is no need for additional frontend technologies like React or HTML/CSS. This is in line with my instructor, Prof Priya's advice to use frameworks selectively for the frontend.

Over a period of two days (January 31 - February 1), I worked through 8 sections of the course that taught: Streamlit basics and components, layout management (columns, sidebars, tabs), data display with st.dataframe and st.table, interactive charts (plotly, altair), map visualizations with folium, ML model deployment with Streamlit and building interactive dashboards. Key concepts I learned were using Streamlit's session_state to store user data across reruns, leveraging st.cache_data for faster data loading, and developing a multi-page app by following the pages/ folder convention.

Proof of Concept: StatCan Dashboard

To check my understanding, I have prepared a proof-of-concept dashboard integrating real data from Statistics Canada (StatCan) to showcase the population trends in Canada. From this work, I showed how I can link up outside datasets, build interactive filter controls and display professional visuals with Streamlit's built-in widget library (e.g., st.line_chart, st.selectbox, st.dataframe).

Population of Canada

Source table can be found [here](#)

> See full data table

Choose a starting date
Quarter
Q1
Year
2013

Choose an end date
Quarter
Q1
Year
2023

Choose a location
Choose a location
British Columbia

Analyze

Population change Compare

Population change from Q1 2013 to Q1 2023

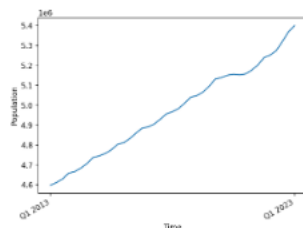
Q1 2013

4598255

Q1 2023

5399118

↑ 17.42%



Population of Canada

Source table can be found [here](#)

> See full data table

Choose a starting date
Quarter
Q1
Year
2013

Choose an end date
Quarter
Q1
Year
2023

Choose a location
Choose a location
British Columbia

Analyze

Population change Compare

Compare with other locations

Choose other locations

British Columbia x New Brunswick x Ontario x Manitoba x Alberta x

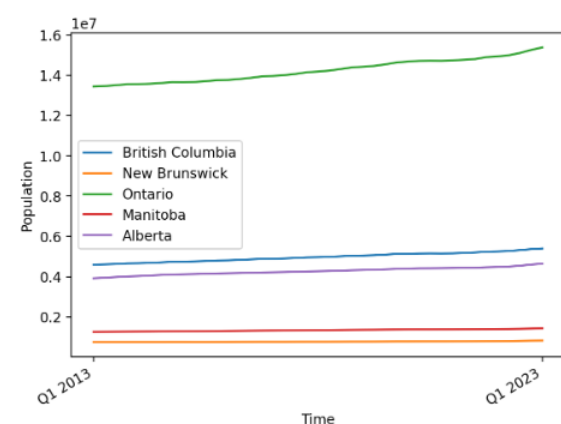


Fig 2.2.1 – Proof of Concept: StatCan Population of Canada Dashboard (Built and run on Streamlit)

Github Link:

https://github.com/desporamon/W26_4495_S2_DesmondC/blob/main/Implementation/samples/2_streamlit_fundamentals/12_main.py

2.3 UI Mockups (Sprint 1A)

Before coding, I made 8 UI mockups with the help of UX Pilot, an AI-assisted design tool, to set up the visual direction and user experience of the platform. A mockup-first approach allows implementation to closely follow a design specification, minimizes rework, and serves as a reference point for testing. Each mockup was done with healthcare aesthetics in mind; the Vancouver Coastal Health (VCH) teal color scheme (#006271) was used to signify trust and professionalism. The mockups depict the entire user journey: authentication (login/register), the home dashboard with health summary and quick actions, the symptom assessment chatbot interface, personal analytics dashboard, facility finder with map, health education library, and system-level analytics. The complete set of 8 UI mockups can be found in Appendix B.

2.4 Database Setup (Sprint 1C)

The application employs SQLite as its database engine. It was a natural choice given SQLite's simplicity, the fact that it requires no configuration, and its appropriateness for a single-user, proof-of-concept kind of application. The database module (components/database.py) is an implementation of two core tables and full CRUD (Create, Read, Update, Delete) functionalities.

Database Schema:

Table	Key Columns	Purpose	CRUD Functions
users	id, name, email, password_hash, age, gender, region, conditions	Store user registration data for authentication	create_user, get_user_by_email, get_user_by_id, verify_user_exists
assessments	id, user_id, symptoms, extracted_symptoms, urgency_level, recommendation	Store chatbot symptom assessment results	save_assessment, get_user_assessments, get_assessment_by_id, get_assessment_stats

The database file (health_platform.db) is stored in the Implementation/data/ directory. During development, I used the SQLite Viewer VS Code extension to visually inspect table contents and verify data integrity.

2.5 Authentication Feature (Sprint 1D)

The authentication system came out to be the most technically challenging component of Sprint 1. It securely registers users and logs them in by bcrypt password hashing, Streamlit session_state management, and page-level access control. The work was divided into three logical batches:

Batch 1 - Password Security (auth.py): The code contains a new authentication module focused solely on password security that uses the bcrypt library. To ensure that every password is hashed with a different salt, the hash_password() function internally calls a method that generates a unique salt before producing an irreversible hash. The verify_password() function takes the user input and verifies it securely against the stored hash without decrypting the original password at any point. Additional validation checks include a minimum password length of 8 characters and a correct email format (must contain an @ symbol).

Key code snippet - Password hashing with bcrypt:

```
import bcrypt
```

```
def hash_password(password: str) -> str:

    salt = bcrypt.gensalt()

    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)

    return hashed.decode('utf-8')


def verify_password(password: str, hashed: str) -> bool:

    return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))
```

Batch 2 - Login & Registration UI: Developed an entire authentication interface utilizing Streamlit tabs. The Login tab has an email input, a password field with a visibility toggle, and a "Remember me" checkbox. The Register tab has several fields for user details: full name, email, password with confirmation, age, gender, BC health region (a dropdown of 5 health authority regions), and 10 checkboxes for health conditions (such as Diabetes, Hypertension, Heart Disease, etc.). The validation of the forms is done on both client-side (field completion) and server-side (email format, password strength, duplicate email check).

Batch 3 - Page Protection: I made a helper function `require_authentication()` that looks up Streamlit `session_state` to check a valid user login. This helper function is called from the very top of each page file. If the user is not logged in, a warning message on the page ("Please login to access this page") is shown and the rendering process is stopped. This is how it is made sure that no one can access any of the platform features without authorization.

Authentication Test Results:

A comprehensive set of 9 functional test cases was executed to verify the authentication system works end-to-end. All tests passed successfully:

#	Test Case	Action	Expected Result	Status
1	New User Registration	Register with valid credentials	Success message, auto-login, home page displayed	PASS
2	Duplicate Email	Register with existing email	Error: 'An account with this email already exists'	PASS
3	Password Validation	Register with weak password ('123')	Error: 'Password must be at least 8 characters long'	PASS
4	Email Validation	Register with invalid email ('notanemail')	Error: 'Email must contain @ symbol'	PASS
5	Login - Correct	Login with registered credentials	Success, redirected to Home page	PASS
6	Login - Wrong Password	Login with incorrect password	Error: 'Incorrect password. Please try again.'	PASS

7	Logout	Click logout button	Session cleared, returned to login page	PASS
8	Page Protection	Access pages while logged out	'Please login to access this page' warning	PASS
9	Database Verification	Check SQLite database records	Password stored as bcrypt hash (\$2b\$12\$...), not plain text	PASS

2.6 Basic Navigation (Sprint 1E)

The last part of Sprint 1 set up the multi-page navigation structure. Six pages files were created in the pages/ directory under Streamlit's naming convention (numbered prefix with emoji for visual identification). All pages were checked to open properly via the sidebar navigation, and a user needs to be logged in for each page as the `require_authentication()` function is called on each one. The pages are: Home, Symptom Assessment, My Dashboard, Facility Finder, Health Library, and System Analytics. At this point, pages consist of placeholder content that will be completely implemented in the subsequent sprints.

2.7 Issues Encountered and Solutions

Issue	Impact	Solution
SQLite3 CLI not available on Windows	Could not verify database contents via command line	Installed SQLite Viewer VS Code extension; also used Python interactive mode to query database
PowerShell escaping issues with Python one-liners	Could not run database cleanup commands directly	Used Python interactive mode (python >>> prompt) to execute commands line by line
Orphaned assessment records after deleting test user	Assessments referenced a non-existent user_id	Manually deleted orphaned records; noted need for foreign key constraints in future
Demo mode credentials visible on login page	Security concern for production deployment	Noted for removal before final submission; acceptable during development phase

3. Repo Check-in of Implementation Completed

GitHub Repository: https://github.com/desporamon/W26_4495_S2_DesmondC

The following files and folders have been checked into the repository since the project began. All work is committed with descriptive commit messages and pushed to the main branch:

File/Folder	Description
Implementation/src/app.py	Main Streamlit application with Login/Register UI
Implementation/src/components/auth.py	Authentication module (bcrypt hashing)
Implementation/src/components/database.py	SQLite database module (CRUD operations)
Implementation/src/components/chatbot.py	Chatbot logic module (placeholder)
Implementation/src/components/openai_utils.py	OpenAI API integration (placeholder)
Implementation/src/components/rules.py	CTAS rule-based safety checks (placeholder)
Implementation/src/pages/1_Home.py	Home page with welcome message
Implementation/src/pages/2_Symptom_Assessment.py	Chatbot page (placeholder)
Implementation/src/pages/3_My_Dashboard.py	Personal dashboard (placeholder)
Implementation/src/pages/4_Facility_Finder.py	Facility finder (placeholder)
Implementation/src/pages/5_Health_Library.py	Health library (placeholder)
Implementation/src/pages/6_System_Analytics.py	System analytics (placeholder)
Implementation/src/.streamlit/config.toml	Theme configuration (VCH teal colors)
Implementation/data/health_platform.db	SQLite database file
Misc/design/	8 UI mockup images (UX Pilot)
README.md	Project documentation with repository structure
requirements.txt	Python package dependencies

Git Commit History:

Jan 30: "Initial project setup - environment, folder structure, requirements.txt"

Feb 2: "Sprint 1A and 1B - UI mockups and project theme configuration"

Feb 3: "Sprint 1C complete - Database setup with users and assessments tables"

Feb 5: "Sprint 1D complete - Authentication feature with bcrypt hashing, cleaned test data"

4. Revised Timeline and Next Steps

Considering the completion of Sprint 0 and Sprint 1, and after receiving comments from the instructor on the proposal, the project schedule has been revised to make sure that a partially working demo is available for the midterm report submission (due February 23, 2026). The new sprint schedule looks like this:

Sprint	Description	Dates	Status
Sprint 0	Environment Setup (Python, VS Code, GitHub, packages)	Jan 30-31	DONE
Sprint 1A	UI Mockups (8 screens using UX Pilot)	Feb 2-3	DONE
Sprint 1B	Project Structure + VCH Theme	Feb 2	DONE
Sprint 1C	Database Setup (SQLite, schemas, CRUD)	Feb 3	DONE
Sprint 1D	Authentication Feature (bcrypt, login/register, protection)	Feb 4-5	DONE
Sprint 1E	Basic Navigation (6 pages verified)	Feb 5	DONE
Sprint 2A	OpenAI API Integration (secrets, extract_symptoms)	Feb 6	Next
Sprint 2B	CTAS Safety Rules (critical symptom detection)	Feb 7	Planned
Sprint 2C	ML Model Training (Random Forest classifier)	Feb 8-9	Planned
Sprint 2D	Chatbot UI + Full Integration Pipeline	Feb 10-13	Planned
Sprint 3A-3D	Home Dashboard, Personal Analytics, Facility Finder, Health Library	Feb 23 - Mar 4	Planned
Sprint 4A-4B	Synthetic Data + System Analytics Dashboard	Mar 5-12	Planned
Sprint 5	Deployment + Testing	Mar 13-20	Planned
Sprint 6	Documentation + Final Submission	Mar 21 - Apr 10	Planned

GANTT CHART - PROJECT TIMELINE													
Phase	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13
	Jan 27	Feb 3	Feb 10	Feb 17	Feb 24	Mar 3	Mar 10	Mar 17	Mar 24	Mar 31	Apr 7	Apr 14	Apr 21
Sprint 0: Environment Setup													
Sprint 1: Basic App Structure													
1A: UI Mockups													
1B: Project Structure													
1C: Database Setup													
1D: Authentication Feature													
1E: Basic Navigation													
Sprint 2: Symptom Chatbot													
2A: Open AI Integration													
2B: Safety Layer - CTAS Rules													
2C: ML Model Training													
2D: Chatbot UI + Integration													
Sprint 3: Features 3A-3D													
3A: Home Dashboard													
3B: My Dashboard - Personal Analytics													
3C: Facility Finder													
3D: Health Education Library													
Sprint 4: System Analytics													
4A: System Analytics Data													
4B: System Analytics Dashboard													
Sprint 5: Deployment & Testing													
5A: Deployment													
5B: Testing													
Sprint 6: Documentation													

Fig 4 - Updated Gantt Chart

4.1 What's Next: Sprint 2 - Symptom Assessment Chatbot

The immediate next phase is Sprint 2 (February 6 - 14), which is about developing the core Symptom Assessment Chatbot. This core feature of the project needs to be done before I go on my cruise break (February 14-23) and before the midterm report deadline (February 23). Sprint 2 is made up of four mini-sprints:

Sprint 2A - OpenAI Integration: Set up secure API key storage with Streamlit secrets, write `openai_utils.py` module containing `extract_symptoms()` function that uses GPT to convert unstructured symptom descriptions in natural language into structured data.

Sprint 2B - CTAS Safety Rules: Implement the rule-based safety layer in `rules.py` with 10-15 critical symptom patterns based on Canadian Triage and Acuity Scale (CTAS) guidelines (e.g., chest pain + shortness of breath triggers immediate 911 recommendation).

Sprint 2C - ML Model Training: Research and write about the ML algorithms previously employed in healthcare triage decision-making as a way of explaining the choice of algorithms (responding to instructor feedback). Get symptom-disease data from Kaggle, make urgency levels, fit a Random Forest model with the target 70% accuracy.

Sprint 2D - Chatbot UI & Integration: Build the complete chatbot interface with `st.chat_message`, integrate the 3-layer pipeline (OpenAI extraction, CTAS safety check, ML classification), add color-coded urgency display, and test with 5+ scenarios.

Additionally, as part of addressing my instructor's feedback, I will conduct background research on existing ML algorithms used in healthcare decision-making. This will include documenting the justification for my choice of Random Forest for classification, K-Means for clustering, and Isolation Forest for anomaly detection, supported by 3-5 academic citations demonstrating prior precedent in health-related applications.

The midterm video demo will showcase: user registration and login, the full symptom assessment flow (user input, OpenAI extraction, safety check, ML classification, color-coded recommendation), and basic page navigation.

5. AI Use Section

5.1 AI Tools Usage Summary

During the implementation of sprints 0 and 1, only a few AI tools were used to help explain technical concepts, figure out framework patterns, and generate code for the typical boilerplate parts. I personally planned all the architecture, came up with database schema design, figured out authentication logic, and testing strategies. Mainly, I used Claude (Anthropic) as an AI helper for clearing up concepts and for speeding up the code generation of the standard pattern (CRUD functions). Code from AI was always checked, tested, and changed before being integrated.

5.2 Table of AI Tools and Specific Use

AI Tool	Version/Account	Specific Use	Value I Added
Claude (Anthropic)	Claude.ai	Understanding Streamlit framework architecture - how it combines frontend and backend	Designed the application architecture myself; Claude helped clarify how session_state management works in Streamlit
Claude (Anthropic)	Claude.ai	Researching SQLite database setup for Python web applications	Independently designed the database schema based on my project requirements; Used Claude to clarify best practices for password storage
Claude (Anthropic)	Claude.ai	Understanding bcrypt password hashing implementation in Python	Implemented the full auth.py module myself; Claude helped explain the salt generation process and verify_password function pattern
Claude (Anthropic)	Claude.ai	Debugging Streamlit session_state for authentication flow	Wrote all authentication logic; Used Claude to troubleshoot why page protection was not redirecting correctly
Claude (Anthropic)	Claude.ai	Testing methodology for authentication features	Designed the 9 test cases myself based on software testing principles from CSIS courses; Claude helped organize the test checklist
Claude Code (Anthropic)	Claude.ai	AI-assisted code generation for database.py	Reviewed all generated code before accepting; Modified code to match my project requirements and naming conventions

5.3 Statement of Originality

All the decisions regarding the project architecture, design of database schema, authentication flow, UI mockup specification, and testing methodology were literally done by me. I have used AI tools at the level of reading documentation or getting clarification from a classmate. The code produced by Claude Code was carefully inspected line-by-line before it was accepted, and changes were done to match the project details, naming conventions, and VCH healthcare theming. The entire prompt history is recorded in the Appendix.

Appendix A: AI Prompt History

The following documents my AI tool interactions during this reporting period. AI was used sparingly to clarify technical concepts and accelerate routine code generation. Each entry shows the context, my prompt, and the value I contributed beyond the AI response.

Prompt Session 1: Understanding Streamlit Session State

Date: January 31, 2026 | Tool: Claude (Anthropic)

Context: While learning Streamlit from the Udemy course, I needed to understand how `session_state` works for maintaining user login status across page navigation.

My Prompt: *"I'm building a multi-page Streamlit app that needs user authentication. How does `st.session_state` work for keeping a user logged in when they navigate between pages?"*

Summary of Response: Claude explained that `session_state` persists across Streamlit reruns and page navigation, making it ideal for storing authentication status. It showed the pattern of checking `session_state` at the top of each page.

Value I Added: I designed the complete authentication flow myself, including which session variables to store (`user_id`, `user_name`, `logged_in`) and how page protection should work. The `session_state` pattern was applied to my specific use case.

Prompt Session 2: SQLite Database Setup for Python

Date: February 3, 2026 | Tool: Claude (Anthropic)

Context: I was designing the database schema for the platform and wanted to confirm best practices for SQLite with Python, specifically around password storage.

My Prompt: *"For my health platform, I need to store user accounts and symptom assessments in SQLite. What is the recommended approach for storing passwords securely? Should I use plain text, MD5, or something else?"*

Summary of Response: Claude recommended `bcrypt` for password hashing as it includes automatic salting and is computationally expensive to brute-force. It advised against MD5 and SHA-256 for passwords as they are too fast to hash.

Value I Added: I independently designed the complete database schema (users and assessments tables) with appropriate columns based on my project requirements. The `bcrypt` recommendation confirmed my initial research. I then implemented the full `database.py` module with all CRUD functions.

Prompt Session 3: Bcrypt Implementation in Python

Date: February 4, 2026 | Tool: Claude (Anthropic)

Context: While implementing auth.py, I needed to understand the specific bcrypt API in Python to implement the hash_password and verify_password functions.

My Prompt: *"How do I implement bcrypt password hashing in Python? I need a hash_password function that takes a string and returns a hash, and a verify_password function that checks a password against a stored hash."*

Summary of Response: Claude provided the bcrypt.gensalt() and bcrypt.hashpw() pattern for hashing, and bcrypt.checkpw() for verification, including the encode/decode steps for string conversion.

Value I Added: I implemented the complete auth.py module with additional validation logic (email format checking, password strength requirements) that was not covered in the AI response. I also integrated the auth functions with my database module and Streamlit session management.

Prompt Session 4: Streamlit Login/Register UI Pattern

Date: February 4, 2026 | Tool: Claude (Anthropic)

Context: I had designed the login/register mockups in UX Pilot and needed to implement them in Streamlit with tabs, form validation, and proper layout.

My Prompt: *"I have a mockup for a login/register page with tabs. How do I create a tabbed interface in Streamlit with form validation? The register form needs to include dropdown selectors and checkboxes for health conditions."*

Summary of Response: Claude explained Streamlit's st.tabs() component and st.form() for grouping inputs with a single submit button, along with st.selectbox for dropdowns and st.checkbox for health conditions.

Value I Added: I designed the entire form layout to match my UX Pilot mockups, selected the specific BC health regions and health conditions relevant to my project, implemented custom CSS for centering and styling, and added comprehensive validation logic for all fields.

Prompt Session 5: Page Protection Pattern

Date: February 5, 2026 | Tool: Claude (Anthropic)

Context: After implementing login, I needed to protect all pages so unauthenticated users cannot access them.

My Prompt: *"How do I protect Streamlit pages so that only logged-in users can access them? I want unauthenticated users to see a message telling them to login."*

Summary of Response: Claude suggested creating a helper function that checks session_state and uses st.warning() plus st.stop() to halt page rendering for unauthenticated users.

Value I Added: I created the require_authentication() function and systematically applied it to all 6 pages. I designed the specific warning messages and added a link back to the login page for better user experience.

Prompt Session 6: Claude Code - Database Module Generation

Date: February 3, 2026 | Tool: Claude Code

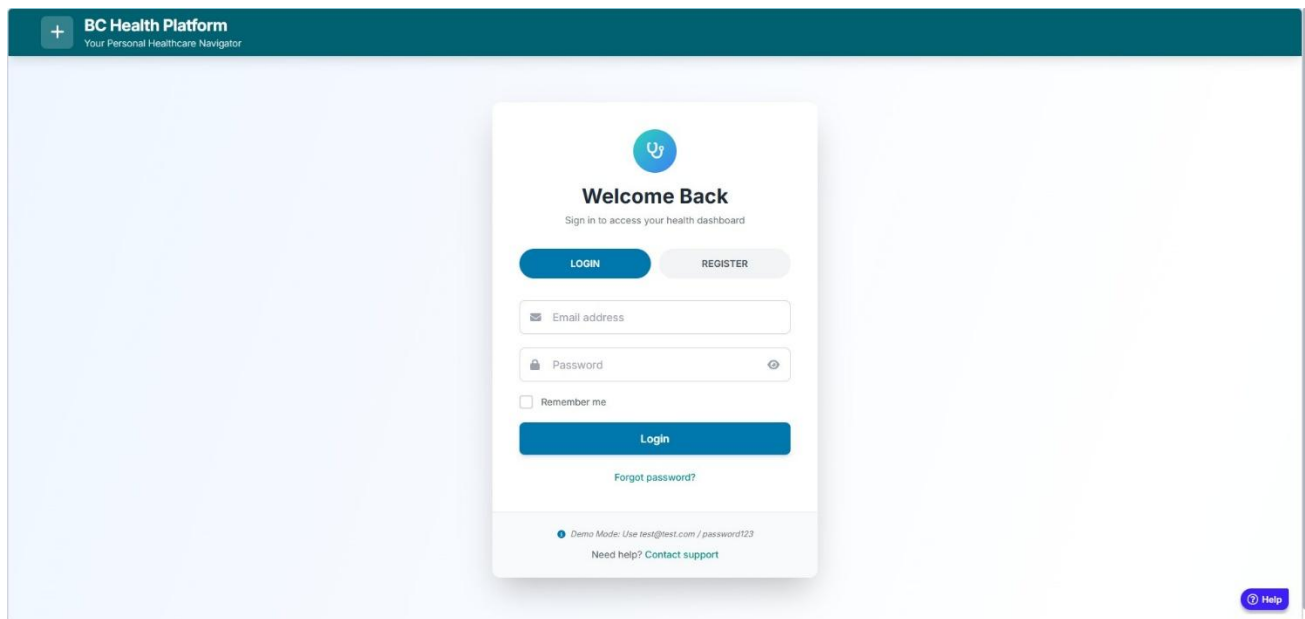
Context: Used Claude Code to accelerate the generation of database.py with SQLite CRUD functions based on my schema design.

My Prompt: *"Create components/database.py for a health platform with SQLite. Tables: users (id, name, email, password_hash, age, gender, region, conditions, created_at) and assessments (id, user_id, symptoms, extracted_symptoms, urgency_level, recommendation, created_at). Include all CRUD functions with proper error handling."*

Summary of Response: Claude Code generated a 690-line database module with table creation, CRUD functions, and test data insertion.

Value I Added: I reviewed the entire generated code, verified the SQL schema matched my design, tested all functions using SQLite Viewer, and cleaned up test data that was no longer needed after Sprint 1D implementation.

Appendix B: UI Mockups



The login page features a dark teal header with the BC Health Platform logo and tagline. A central white card contains the login form. The card has a blue circular icon with a white 'U' and 'H' at the top. Below the icon, the text 'Welcome Back' is followed by 'Sign in to access your health dashboard'. There are two buttons: 'LOGIN' in blue and 'REGISTER' in grey. Below these are input fields for 'Email address' and 'Password'. A 'Remember me' checkbox is present. A blue 'Login' button is at the bottom of the card. Below the button is a link for 'Forgot password?'. At the very bottom of the card, there is a demo mode notice and a link for 'Need help? Contact support'. A small 'Help' button is in the bottom right corner of the page.

BC Health Platform
Your Personal Healthcare Navigator

Welcome Back
Sign in to access your health dashboard

LOGIN REGISTER

Email address

Password

☐ Remember me

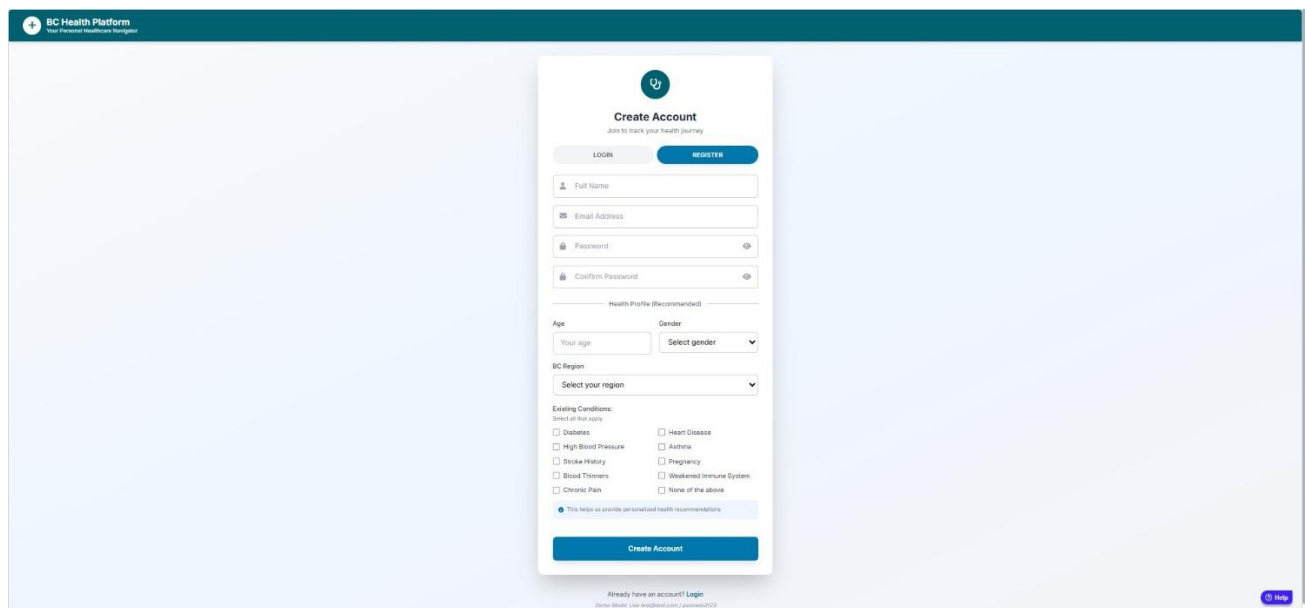
Login

[Forgot password?](#)

Demo Mode: Use test@test.com / password123
[Need help? Contact support](#)

Help

Fig B1 – Login Page



The register page features a dark teal header with the BC Health Platform logo and tagline. A central white card contains the registration form. The card has a blue circular icon with a white 'U' and 'H' at the top. Below the icon, the text 'Create Account' is followed by 'Join to track your health journey'. There are two buttons: 'LOGIN' in grey and 'REGISTER' in blue. Below these are input fields for 'Full Name', 'Email Address', 'Password', and 'Confirm Password'. Below these are sections for 'Health Profile (Recommended)' with fields for 'Age', 'Gender', and 'BC Region'. There is a section for 'Existing Conditions' with checkboxes for various conditions. A blue 'Create Account' button is at the bottom of the card. Below the button is a link for 'Already have an account? Login'. At the very bottom of the card, there is a demo mode notice. A small 'Help' button is in the bottom right corner of the page.

BC Health Platform
Your Personal Healthcare Navigator

Create Account
Join to track your health journey

LOGIN REGISTER

Full Name

Email Address

Password

Confirm Password

Health Profile (Recommended)

Age: Your age Gender: Select gender

BC Region: Select your region

Existing Conditions:
Select all that apply

☐ Diabetes ☐ Heart Disease
☐ High Blood Pressure ☐ Asthma
☐ Stroke History ☐ Pregnancy
☐ Blood Thinners ☐ Weakened Immune System
☐ Chronic Pain ☐ None of the above

☒ This helps us provide personalized health recommendations

Create Account

Already have an account? Login
Demo Mode: Use test@test.com / password123

Help

Fig B2 – Register Page

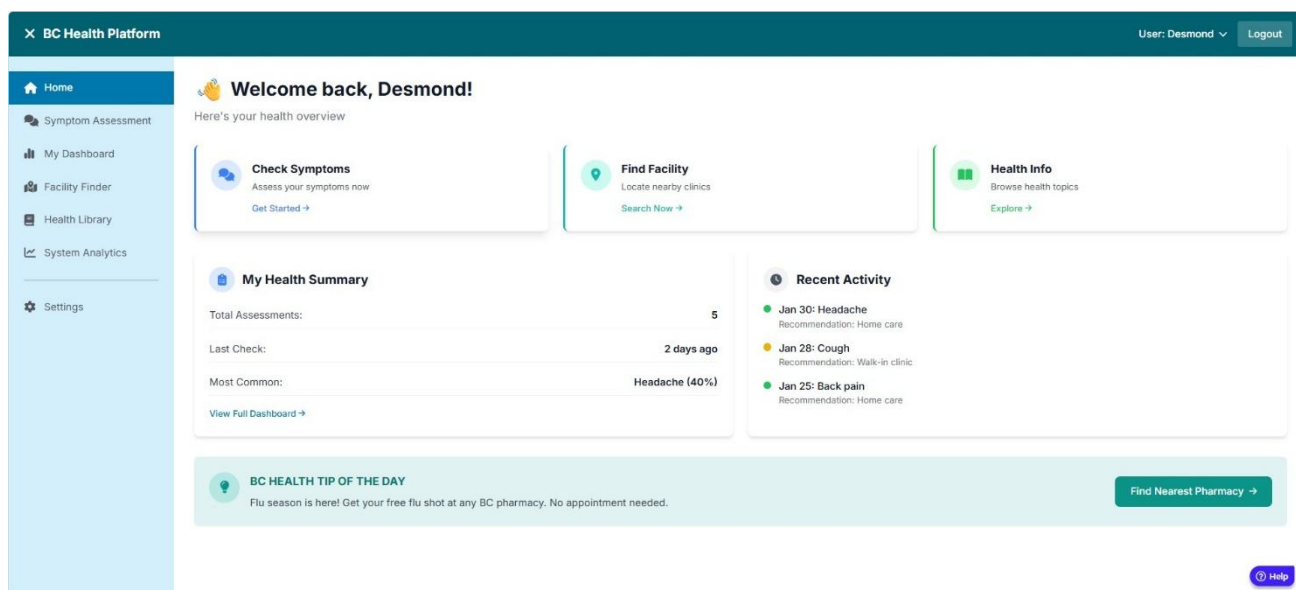


Fig B3 – Home Page

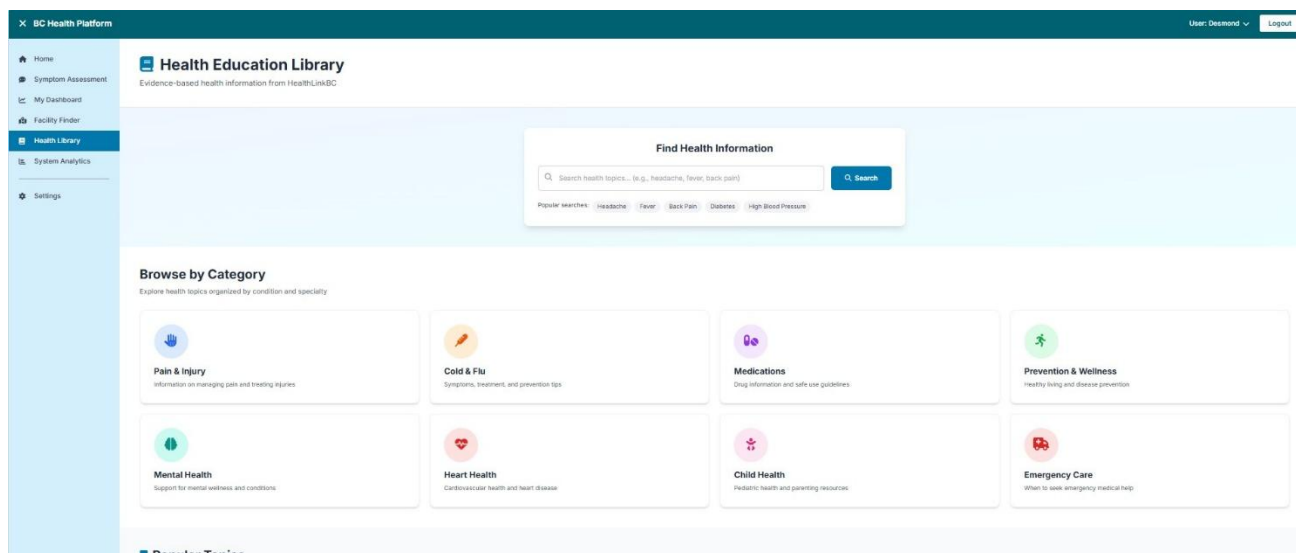


Fig B4 – Health Education Library One

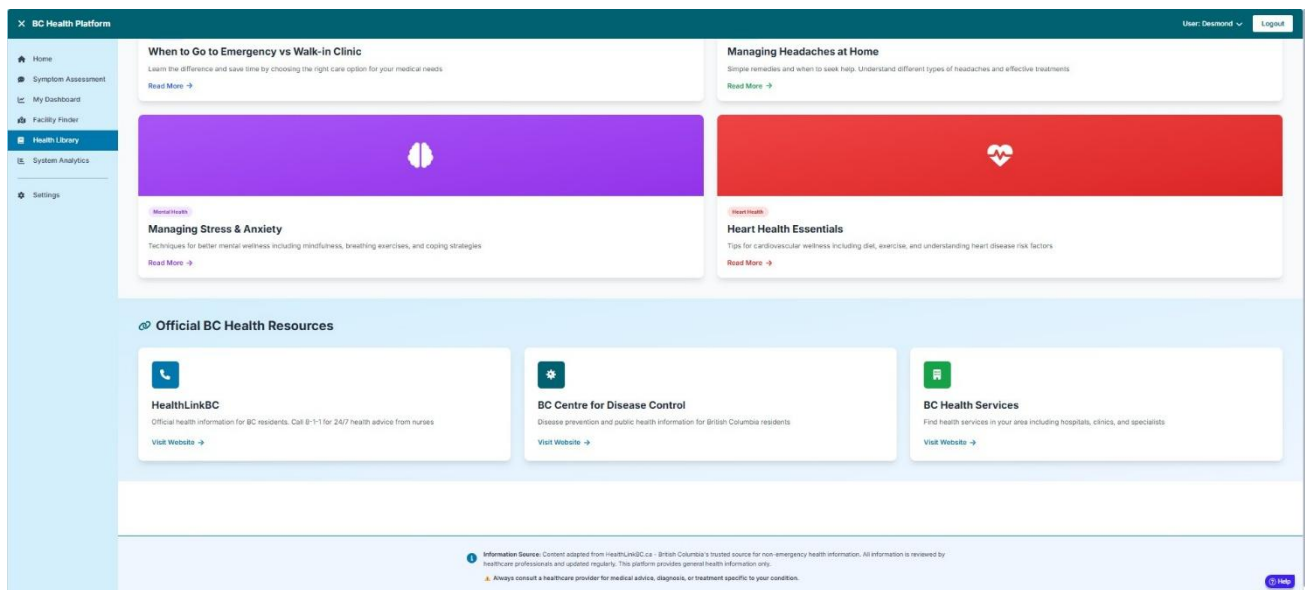


Fig B4 – Health Education Library Two

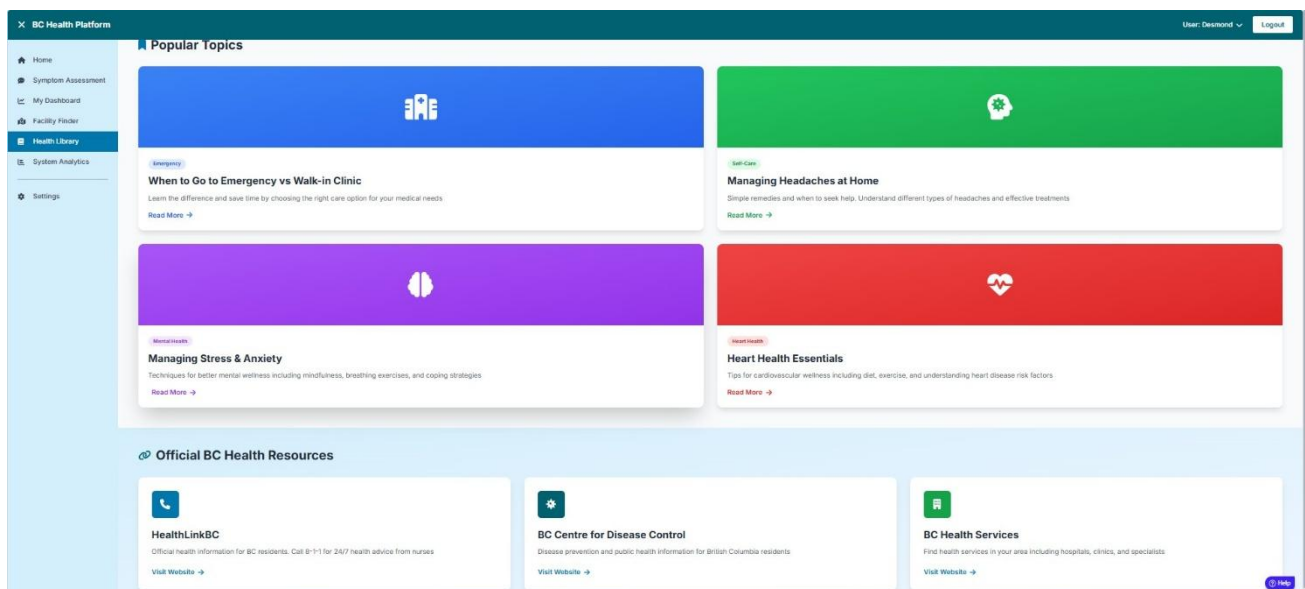


Fig B5 – Health Education Library Three

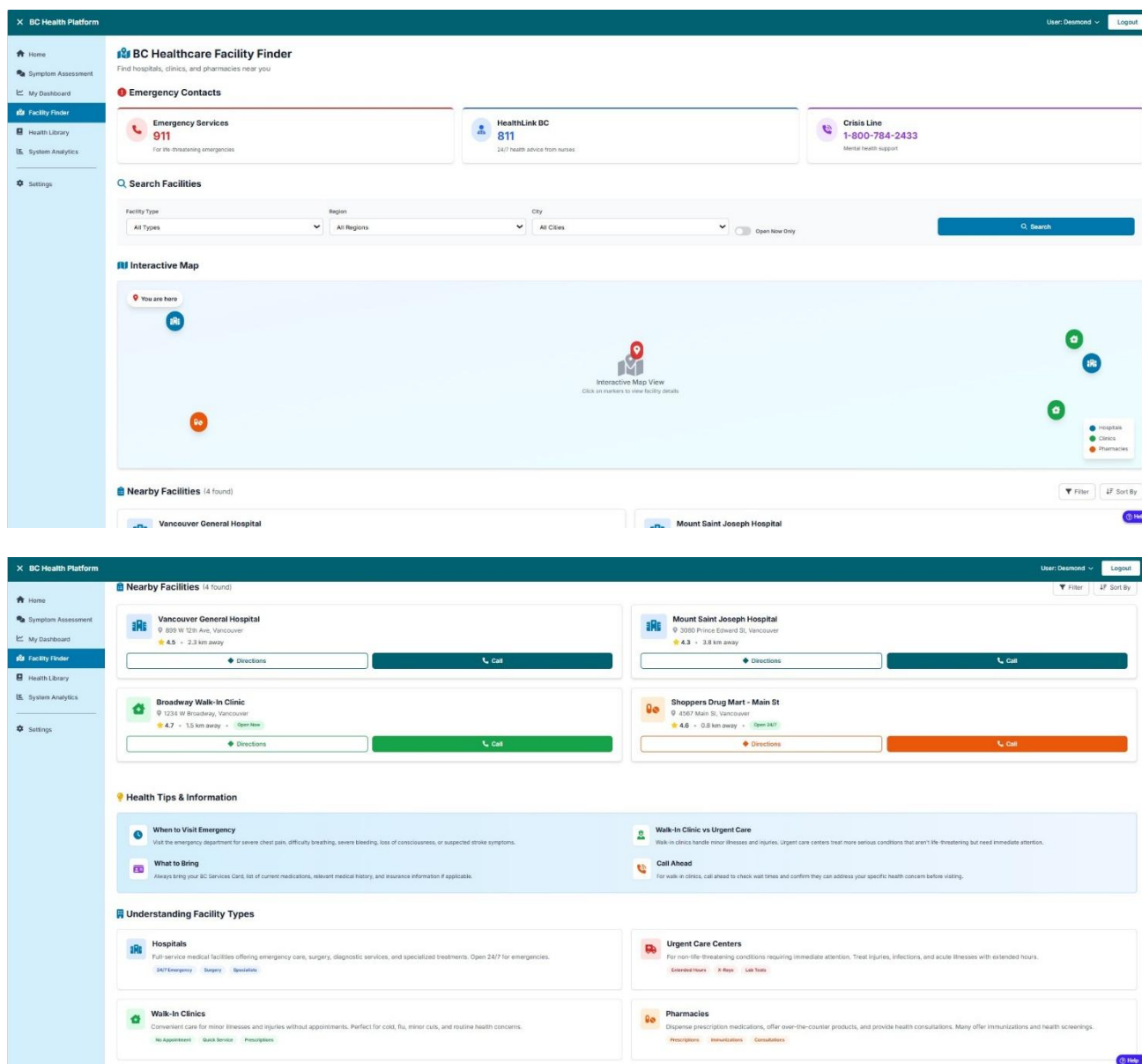


Fig B6 – Facility Finder

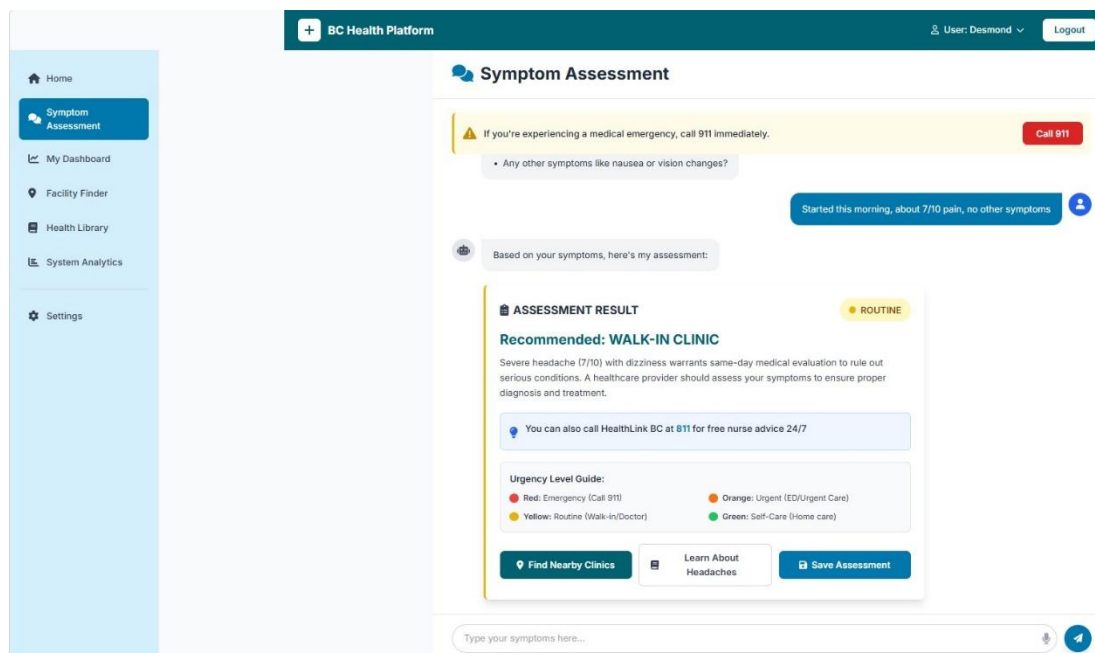
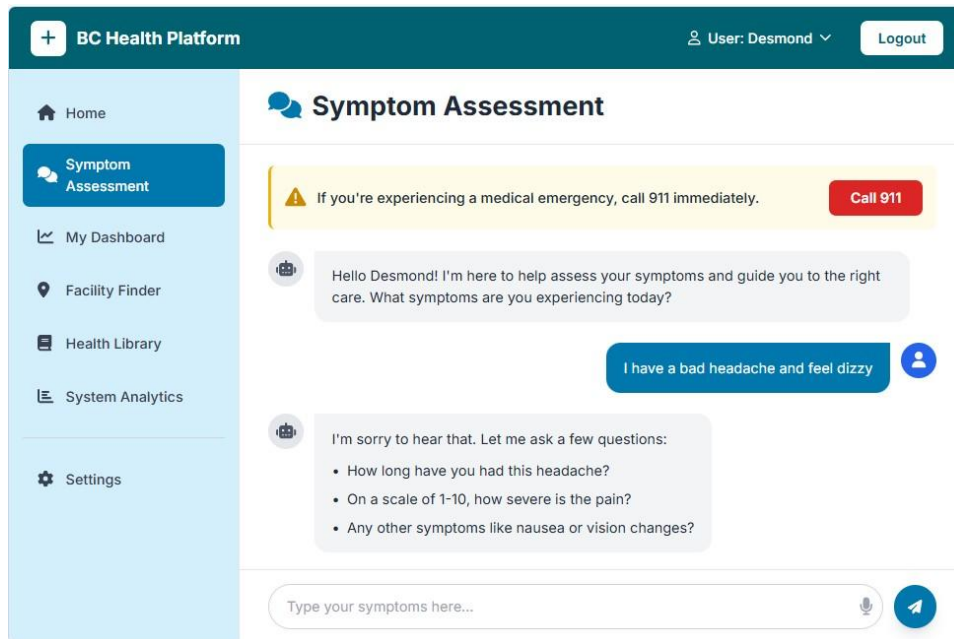


Fig B7 – Symptom Assessment Chatbot Two

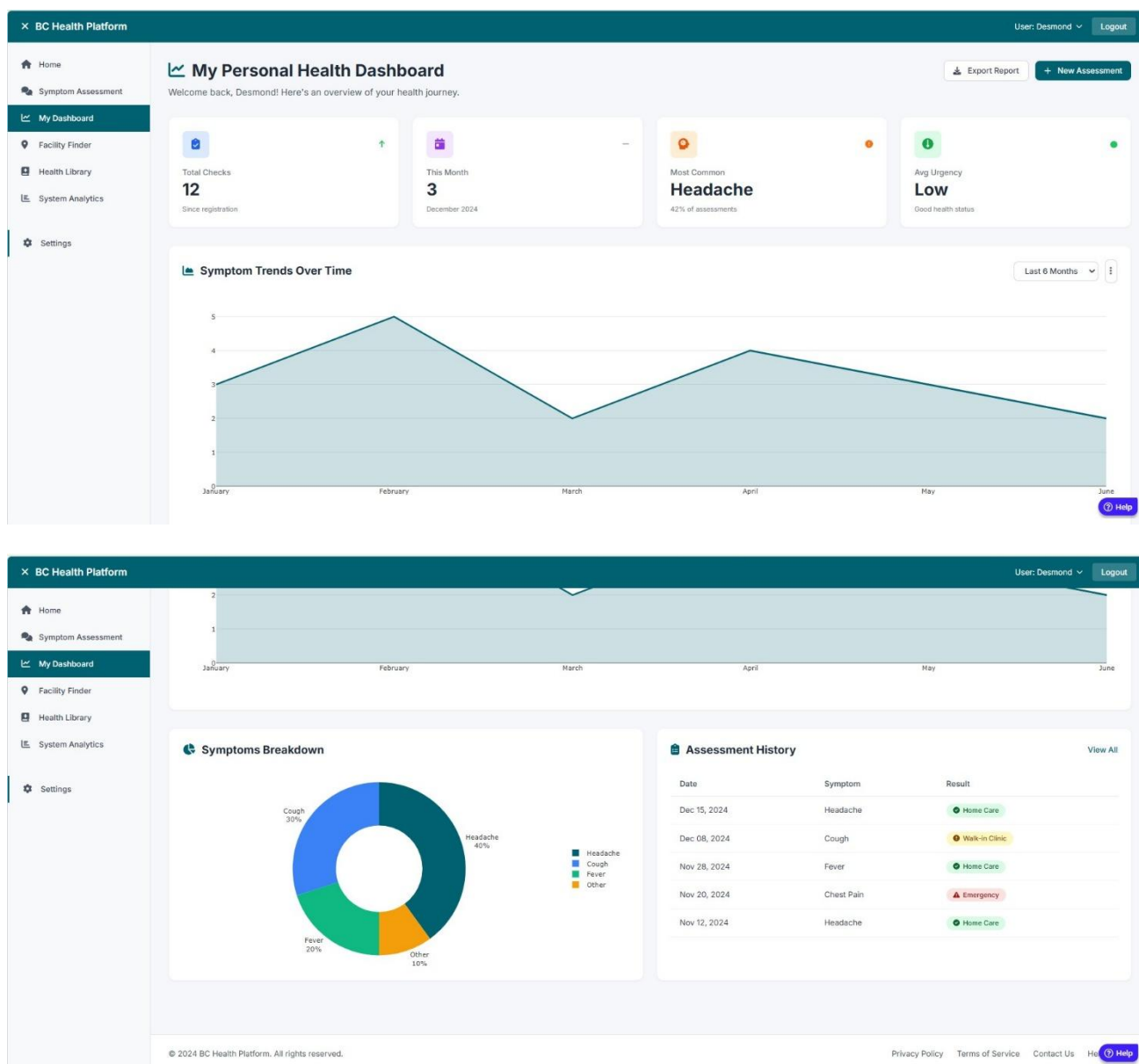


Fig B8 – Personal Health Dashboard

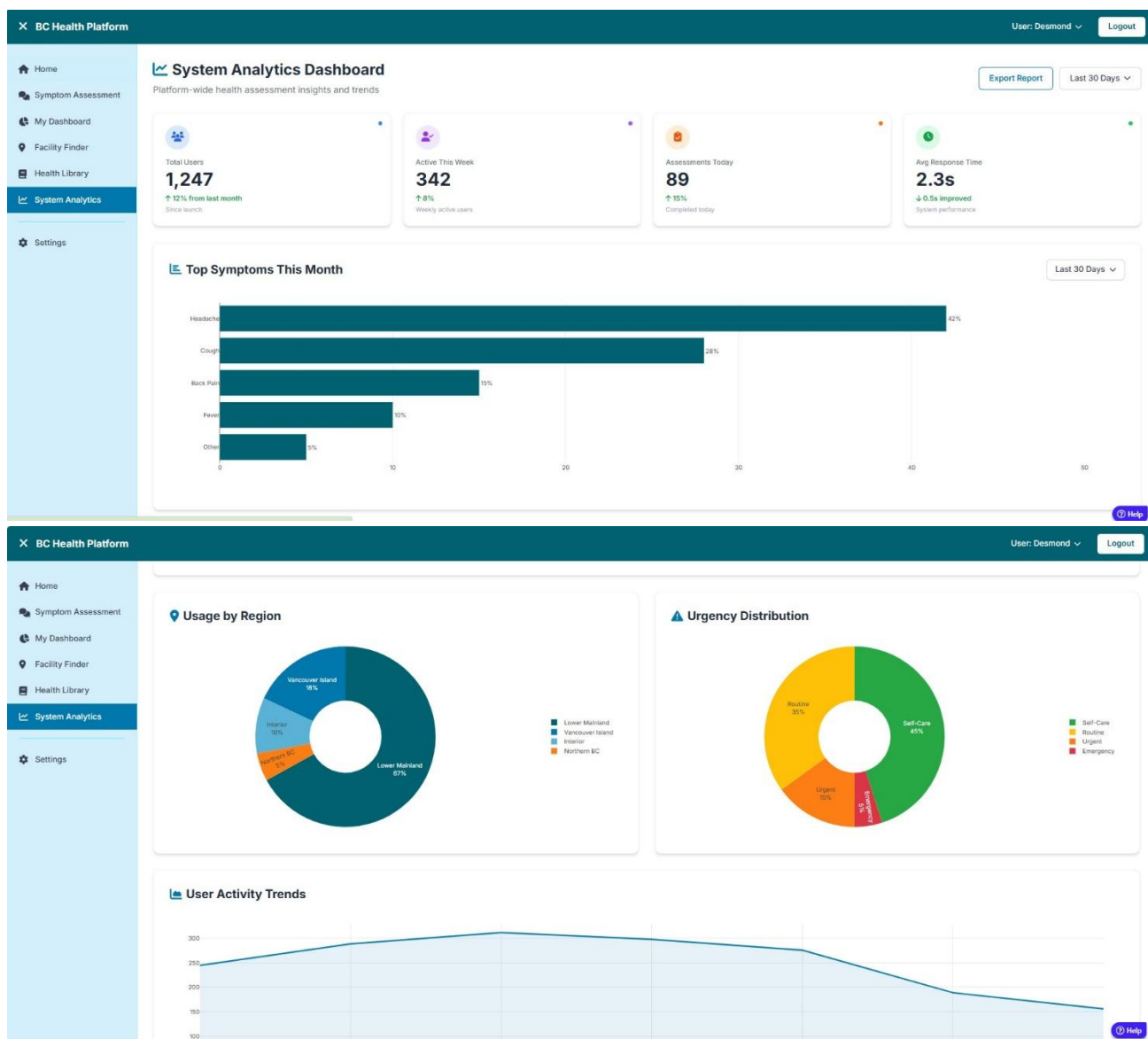


Fig B9 – System Analytics Dashboard